实验报告：请求页式存储管理模拟程序

实验内容

编写一个请求页式存储管理模拟程序，通过对页面置换过程的模拟，加深对请求页式存储管理方式基本原理及实现过程的理解。

要求：

1. 从键盘输入页面访问序列及分配给进程的内存块数；
2. 分别采用 OPT、FIFO 和 LRU 算法进行页面置换（对于 OPT 算法，在有多个页面可选的情况下，先淘汰较早进入的页面）；
3. 计算缺页次数及缺页率。

1. 程序中使用的数据结构及符号说明

数据结构

- `vector<int> pages`： 存储页面访问序列，表示需要访问的页面编号。
- `vector<int> frames`： 存储当前内存块中装入的页面，用于 OPT 和 FIFO 算法。
- `queue<int> fifoQueue`： 用于 FIFO 算法，记录页面进入内存的顺序。
- `vector<int> framesList`： 用于 LRU 算法，记录页面使用的顺序（最近最少使用的页面在最前面，最近使用的页面在最后面）。

符号说明

- `algo`： 算法选择，1 表示 OPT，2 表示 FIFO，3 表示 LRU。
- `frameCount`： 分配给进程的内存块数。
- `pages`： 页面访问序列。
- `pageFaults`： 缺页次数。
- `outputEntries`： 存储每次页面访问后内存块状态和是否命中的信息。

2. 各种调度算法的处理流程，重要模块的详细设计及功能和接口说明

OPT（Optimal）算法

处理流程

1. 对于每个页面访问，检查页面是否已在内存中。
2. 如果命中，记录命中信息。
3. 如果未命中：
    - 检查内存中是否有空闲块。
    - 若有空闲块，将页面装入空闲块。
    - 若无空闲块，选择未来最远才会被访问的页面进行置换（若有多个页面未来访问距离相同，选择最早进入的页面）。

重要模块

- **页面置换逻辑**：
    - **功能**：计算每个页面下一次访问的距离，选择未来最远访问的页面进行置换。
    - **设计**：通过遍历未来页面序列，找到最优置换页面。

FIFO（First-In-First-Out）算法

处理流程

1. 对于每个页面访问，检查页面是否已在内存中。
2. 如果命中，记录命中信息。
3. 如果未命中：
    - 检查内存中是否有空闲块。
    - 若有空闲块，将页面装入空闲块并记录进入顺序。
    - 若无空闲块，置换最早进入的页面。

重要模块

- **队列管理**：
    - **功能**：使用队列记录页面进入内存的顺序，确保最早进入的页面最先被置换。
    - **设计**：通过 `queue<int>` 实现先进先出的页面管理。

LRU（Least Recently Used）算法

处理流程

1. 对于每个页面访问，检查页面是否已在内存中。

2. 如果命中，将该页面移到列表末尾（表示最近使用）。
3. 如果未命中：
    - 检查内存中是否有空闲块。
    - 若有空闲块，将页面装入空闲块并移到列表末尾。
    - 若无空闲块，置换列表最前面的页面（最近最少使用），新页面移到列表末尾。

重要模块

- **列表管理**：
    - **功能**：记录页面使用顺序，最近使用的页面在列表末尾。
    - **设计**：使用 `vector<int>` 模拟双向链表管理页面顺序。

3. 源程序并附注释

以下是完整的 C++ 源程序，包含 OPT、FIFO 和 LRU 算法的实现。

```cpp
#include <bits/stdc++.h>
using namespace std;

// Helper to split a comma-separated string into integers
vector<int> parse_sequence(const string &s) {
    vector<int> seq;
    string num;
    for (char c : s) {
        if (c == ',') {
            if (!num.empty()) {
                seq.push_back(stoi(num));
                num.clear();
            }
        } else if (!isspace(c)) {
            num.push_back(c);
        }
    }
    if (!num.empty()) {
        seq.push_back(stoi(num));
    }
    return seq;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int algo;                // 1 = OPT, 2 = FIFO, 3 = LRU
    int frameCount;          // number of frames
    string seqLine;          // comma-separated page sequence

    // Read inputs
    if (!(cin >> algo)) return 0;
    cin >> frameCount;
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    getline(cin, seqLine);

    vector<int> pages = parse_sequence(seqLine);
    int n = pages.size();

    vector<string> outputEntries;
    int pageFaults = 0;

    if (algo == 1) {
        // OPTIMAL replacement
        vector<int> frames(frameCount, -1);
```

```
vector<int> insertTime(frameCount, -1);
int currentTime = 0;

for (int i = 0; i < n; i++) {
    int p = pages[i];
    bool hit = false;
    for (int f = 0; f < frameCount; f++) {
        if (frames[f] == p) {
            hit = true;
            break;
        }
    }
    if (!hit) {
        pageFaults++;
        int emptyIdx = -1;
        for (int f = 0; f < frameCount; f++) {
            if (frames[f] == -1) {
                emptyIdx = f;
                break;
            }
        }
        if (emptyIdx != -1) {
            frames[emptyIdx] = p;
            insertTime[emptyIdx] = currentTime++;
        } else {
            int victim = 0, farthestNextUse = -1;
            for (int f = 0; f < frameCount; f++) {
                int nextUse = n + 1;
                for (int k = i + 1; k < n; k++) {
                    if (pages[k] == frames[f]) {
                        nextUse = k;
                        break;
                    }
                }
                if (nextUse > farthestNextUse) {
                    farthestNextUse = nextUse;
                    victim = f;
                } else if (nextUse == farthestNextUse) {
                    if (insertTime[f] < insertTime[victim]) {
                        victim = f;
                    }
                }
            }
            frames[victim] = p;
            insertTime[victim] = currentTime++;
        }
```

```
        }
        string entry;
        for (int f = 0; f < frameCount; f++) {
            if (frames[f] == -1) {
                entry.push_back('-');
            } else {
                entry += to_string(frames[f]);
            }
            if (f != frameCount - 1) entry.push_back(',');
        }
        entry.push_back(',');
        entry.push_back(hit ? '1' : '0');
        outputEntries.push_back(entry);
    }

} else if (algo == 2) {
    // FIFO replacement
    vector<int> frames(frameCount, -1);
    queue<int> fifoQueue;
    int currentTime = 0;

    for (int i = 0; i < n; i++) {
        int p = pages[i];
        bool hit = false;
        for (int f = 0; f < frameCount; f++) {
            if (frames[f] == p) {
                hit = true;
                break;
            }
        }
        if (!hit) {
            pageFaults++;
            int emptyIdx = -1;
            for (int f = 0; f < frameCount; f++) {
                if (frames[f] == -1) {
                    emptyIdx = f;
                    break;
                }
            }
            if (emptyIdx != -1) {
                frames[emptyIdx] = p;
                fifoQueue.push(emptyIdx);
            } else {
                int victim = fifoQueue.front();
                fifoQueue.pop();
                frames[victim] = p;
```

```
                fifoQueue.push(victim);
            }
        }
        string entry;
        for (int f = 0; f < frameCount; f++) {
            if (frames[f] == -1) {
                entry.push_back('-');
            } else {
                entry += to_string(frames[f]);
            }
            if (f != frameCount - 1) entry.push_back(',');
        }
        entry.push_back(',');
        entry.push_back(hit ? '1' : '0');
        outputEntries.push_back(entry);
    }

} else if (algo == 3) {
    // LRU replacement using an ordered list/vector
    vector<int> framesList;  // will hold up to frameCount pages, in order from LRU (front

    for (int i = 0; i < n; i++) {
        int p = pages[i];
        bool hit = false;
        // Check if p is already in framesList
        for (auto it = framesList.begin(); it != framesList.end(); ++it) {
            if (*it == p) {
                hit = true;
                // Move this page to the back (most recently used)
                framesList.erase(it);
                framesList.push_back(p);
                break;
            }
        }
        if (!hit) {
            pageFaults++;
            if ((int)framesList.size() < frameCount) {
                // Still space: just append
                framesList.push_back(p);
            } else {
                // Evict the front (least recently used)
                framesList.erase(framesList.begin());
                framesList.push_back(p);
            }
        }
        // Build output entry: print contents of framesList in order, then '-' for empty s
```

```
            string entry;
            for (int f = 0; f < frameCount; f++) {
                if (f < (int)framesList.size()) {
                    entry += to_string(framesList[f]);
                } else {
                    entry.push_back('-');
                }
                if (f != frameCount - 1) entry.push_back(',');
            }
            entry.push_back(',');
            entry.push_back(hit ? '1' : '0');
            outputEntries.push_back(entry);
        }

    } else {
        // Invalid algorithm choice
        return 0;
    }

    // Print the slash-separated sequence
    for (size_t i = 0; i < outputEntries.size(); i++) {
        cout << outputEntries[i];
        if (i + 1 < outputEntries.size()) {
            cout << '/';
        }
    }
    cout << "\n";

    // Print total page faults
    cout << pageFaults << "\n";

    return 0;
}
```

4. 采用的测试方法，对测试结果的分析，以及错误的分析

测试方法

- **输入格式**：
  - 算法选择（1 表示 OPT，2 表示 FIFO，3 表示 LRU）
  - 内存块数
  - 页面访问序列（逗号分隔，例如 "1,2,3,1,4"）

- **输出格式**：
  - 每次页面访问后内存块的状态和是否命中（以斜杠分隔，例如 "1,-,0/1,2,0/..."）
  - 总缺页次数

测试结果分析

- **OPT 算法**：通过预测未来访问选择最优置换，缺页率通常最低，但需要未来访问信息，实际不可行。
- **FIFO 算法**：实现简单，但可能因 Belady 异常导致缺页率随内存块增加而上升。
- **LRU 算法**：基于历史访问记录，性能较好，接近 OPT，但需要额外空间记录访问顺序。

错误分析

- **OPT 算法**：依赖未来信息，实际系统中无法实现，仅用于理论分析。
- **FIFO 算法**：可能置换掉未来频繁使用的页面，导致性能下降。
- **LRU 算法**：需要维护页面访问历史，计算和空间开销较大。

5. 本次实验经验及体会

经验

- 通过模拟页面置换算法，深入理解了虚拟内存管理的核心机制。
- 学会使用队列（FIFO）、向量（LRU、OPT）等数据结构实现不同置换策略。
- 掌握了如何设计程序并通过测试验证算法正确性。

体会

- 页面置换算法的选择显著影响系统性能，不同场景需选用不同算法。
- 实验过程加深了对操作系统内存管理的理解，尤其是页面置换的复杂性。
- 编程实现算法的过程中，体会到理论与实践结合的重要性。